

Stage 1.4.2.4 — OCR Fundamentals

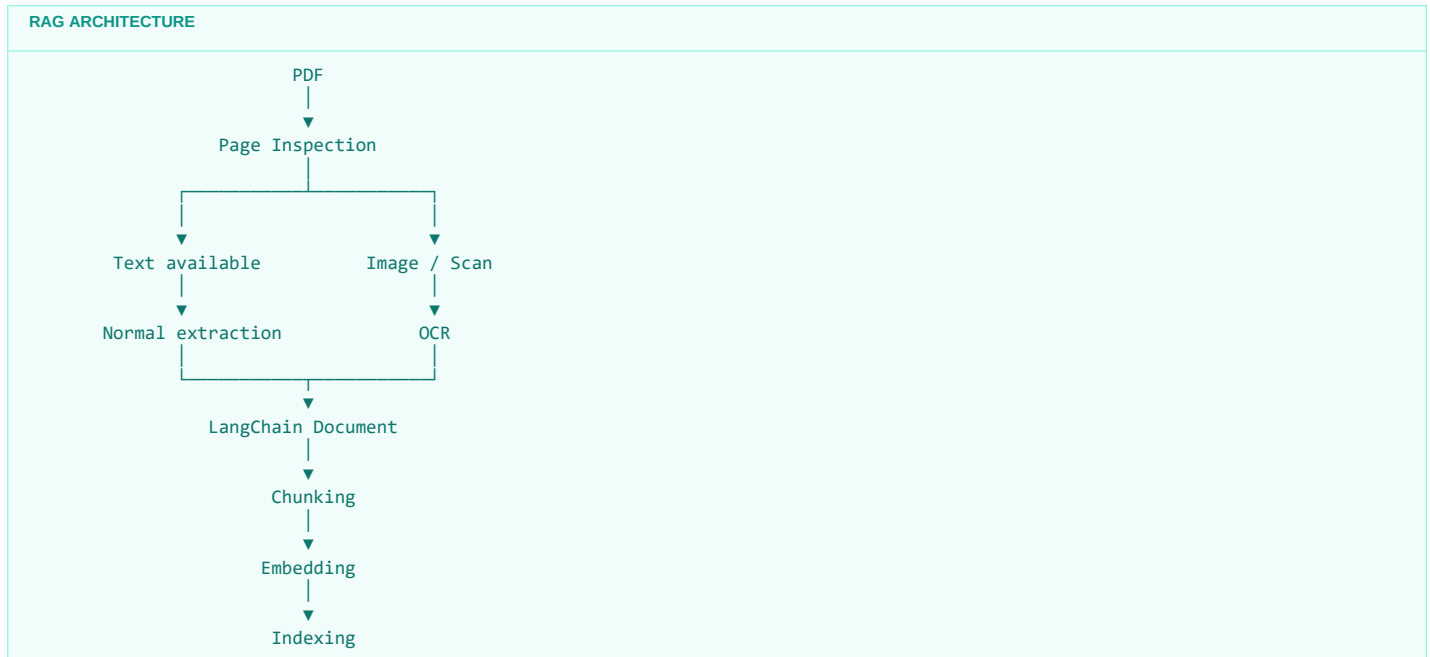
Absolutely. We now move to **Stage 1.4.2.4 — OCR Fundamentals**.

We'll continue our notebook-first approach, and importantly: **We will understand OCR first. We will not install an OCR engine yet.**

The purpose of this stage is to understand exactly what happens between a scanned PDF page and the text that eventually becomes a LangChain Document.

1. Where OCR fits in our RAG pipeline

So far we've reached this point in our architecture:

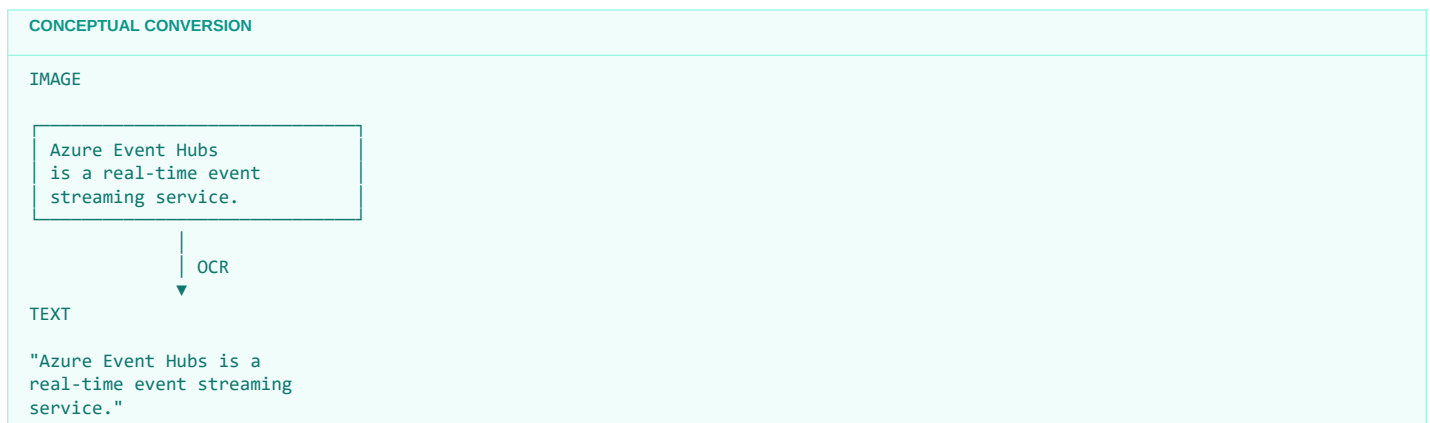


OCR belongs specifically on the **image** → **text** path.

2. What does OCR actually mean?

OCR = Optical Character Recognition

In simple terms: *OCR is technology that looks at an image containing characters and attempts to convert those visual characters into machine-readable text.*



The important point is that OCR isn't simply 'reading a PDF.' It is primarily: **Image** → **Visual character recognition** → **Text**

3. OCR vs PDF text extraction

This distinction is extremely important for RAG:

EXTRACTION MECHANISMS

Normal PDF text extraction

PDF
├── Text objects

PDF parser

Text

OCR

PDF

├── Image

Render/image

OCR

Text

So: PDF extraction and OCR solve completely different problems.

4. Let's demonstrate the difference conceptually

We already have our scanned PDF: `azure_event_hubs_scanned.pdf`.

Visually it contains:

VISUAL CONTENT

Azure Event Hubs

Azure Event Hubs is a managed
real-time event streaming service.

Partitions provide ordered sequences
of events...

But PyMuPDF reported: Characters: 0, Text blocks: 0, Images: 1.

Why? Because internally it is closer to:

INTERNAL STRUCTURE

PDF

├── Page

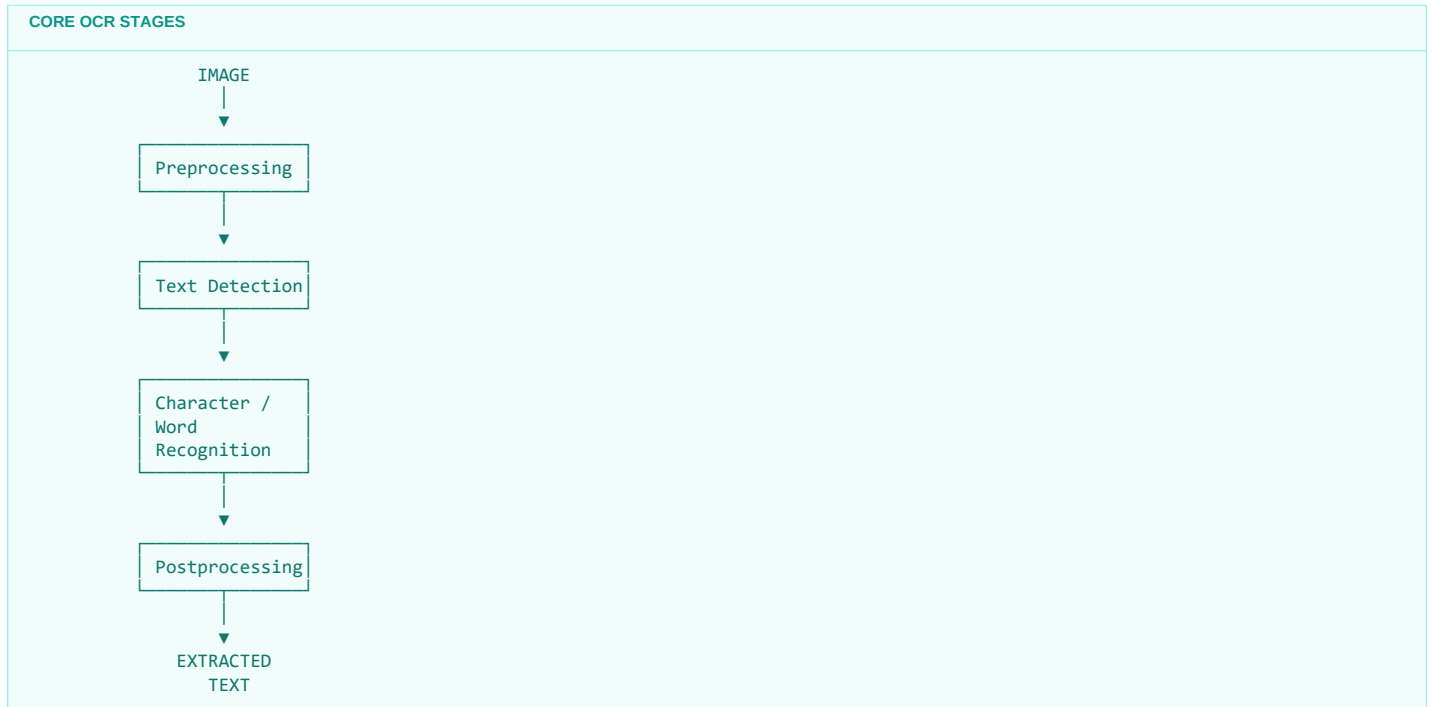
├── Image

├── Pixels

There isn't an actual text layer containing Azure Event Hubs.

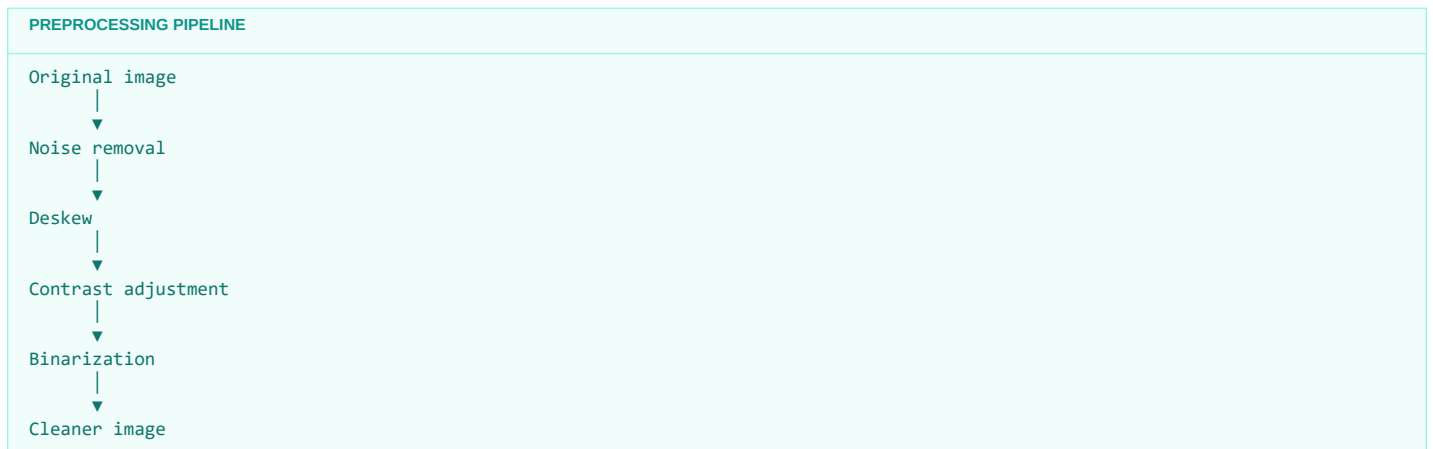
5. The fundamental OCR pipeline

An OCR system conceptually performs the following steps:

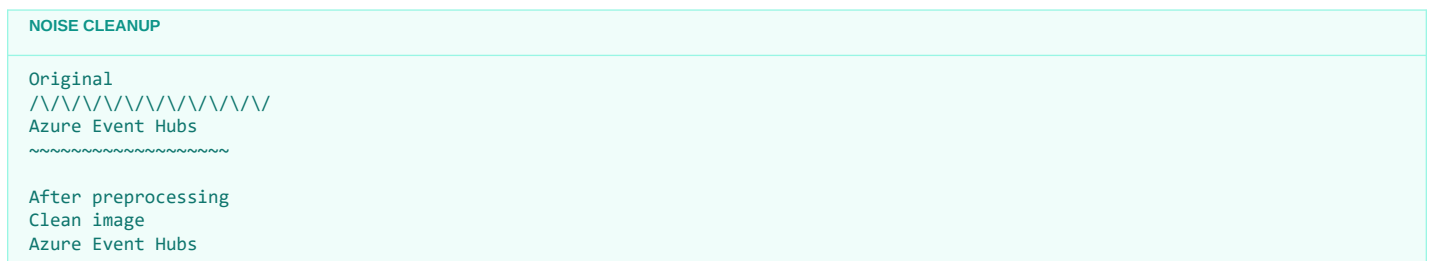


6. OCR Step 1 — Image preprocessing

Before recognition, the image may be cleaned and optimized:



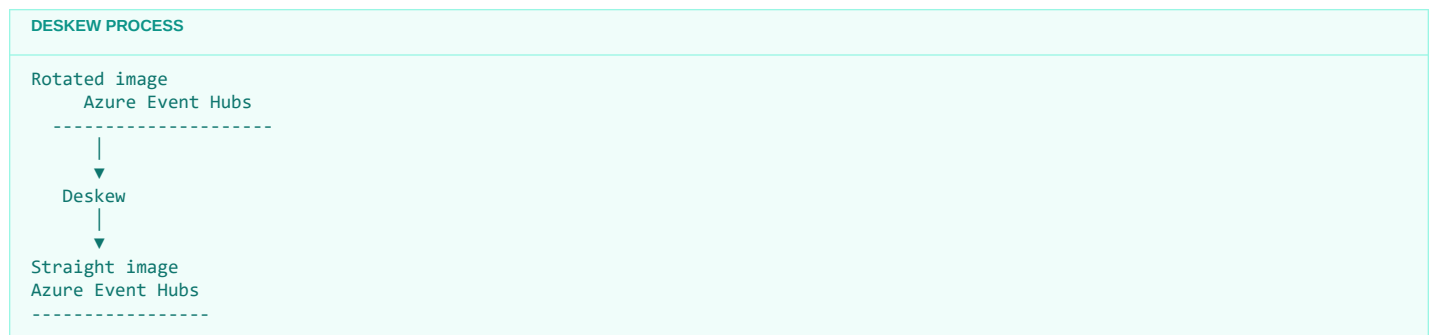
Why? Imagine a noisy document scan:



Recognition becomes significantly easier and more accurate.

7. What is deskewing?

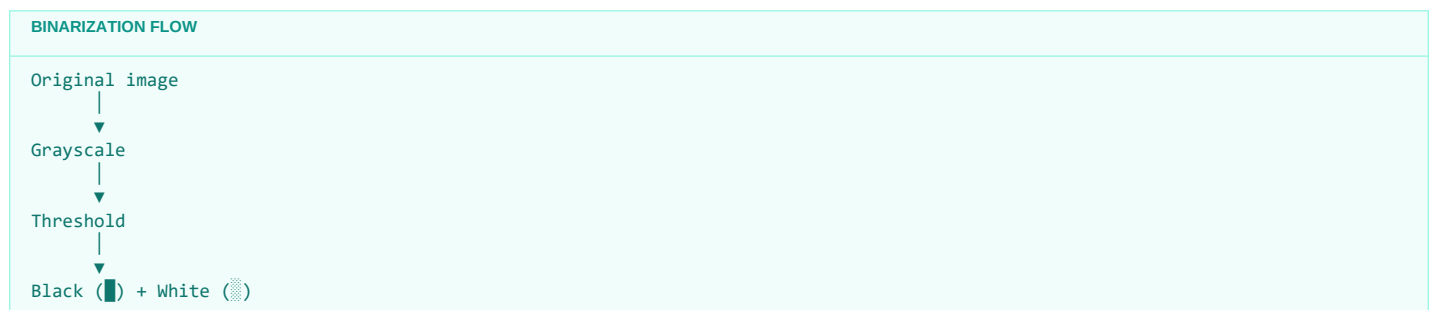
A scanned page might be slightly rotated or tilted:



Deskewing straightens the document to align text lines horizontally.

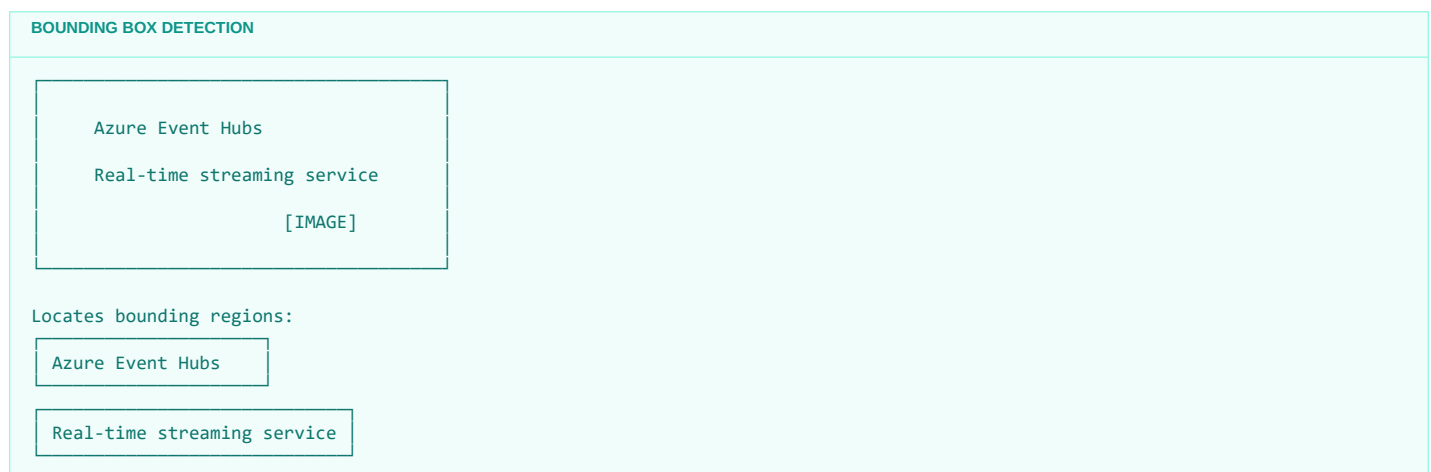
8. What is binarization?

A document image may contain many shades: white, gray, dark gray, and black. Binarization simplifies the image into pure high-contrast values:



9. OCR Step 2 — Text detection

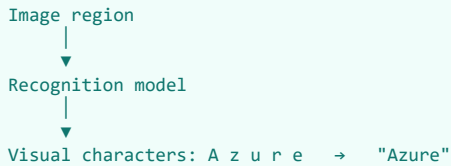
The OCR engine needs to determine: *'Where on the page is text located?'*



10. OCR Step 3 — Text recognition

Once text regions are bounded, the recognition model predicts character representations:





11. OCR doesn't just recognize individual characters

Modern OCR systems generally work with structural hierarchies:

STRUCTURAL HIERARCHY



This matters tremendously for RAG because document structure and semantic boundaries matter.

12. OCR Step 4 — Postprocessing

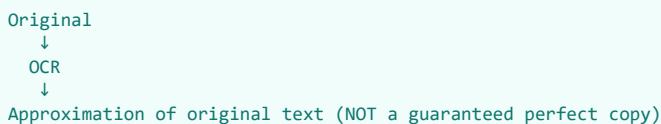
OCR output isn't always perfect. The image may say Azure Event Hubs while OCR produces Azure Event Hub5 or Azure EventHub5.

Postprocessing helps clean whitespace, line breaks, punctuation, and structural alignment without distorting facts.

13. OCR output is not guaranteed to be correct

OCR is probabilistic.

PROBABILISTIC NATURE

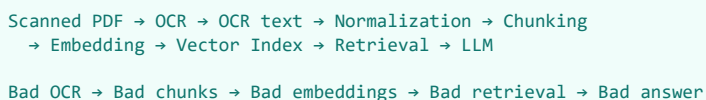


For example: Error code: EH-500 recognized as Error code: EH-SOO could cause downstream vector search to miss critical incidents.

14. Why OCR quality matters to RAG

Let's connect this directly to our RAG retrieval path:

RAG IMPACT CHAIN



Therefore: OCR quality directly affects RAG retrieval accuracy.

15. OCR and layout

Consider a two-column page:

TWO-COLUMN SCAN

Column 1	Column 2
Azure Event Hubs provides...	Consumer groups...

Naïve OCR may read across lines horizontally: Azure Consumer Event groups Hubs provides....

OCR success $\neq$ document extraction success.

16. OCR and tables

Consider a structured tabular scan:

TABLE SCAN

Service	Region	Status
Event Hubs ADX	East US Central	Active Active

Unstructured OCR flattens this to: Service Region Status Event Hubs East US Active ADX Central Active, losing row/column relationships entirely.

17. OCR vs document intelligence

OCR VS INTELLIGENCE

```

OCR
├── "What text is visible?"

Document Intelligence
├── What text?
├── Where is it?
├── Which paragraph?
├── Which table?
├── Which column?
├── Which heading?
├── Which field/value?
└── Which relationship?

```

18. OCR output can contain metadata

Modern OCR systems produce spatial coordinates, bounding boxes, and confidence statistics alongside raw text:

METADATA PAYLOAD

```

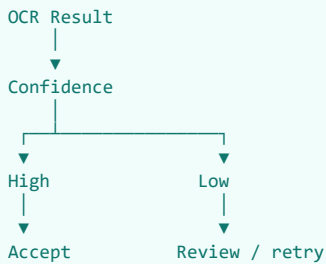
Text: "Azure Event Hubs"
Bounding box: x = 100, y = 200, width = 350, height = 60
Confidence: 0.98

```

19. OCR confidence

Confidence scores allow pipelines to automate routing between straight-through indexing and human review:

CONFIDENCE ROUTING



20. Important distinction: OCR does not create embeddings

OCR and RAG are separate concerns:

SEPARATION OF CONCERNS

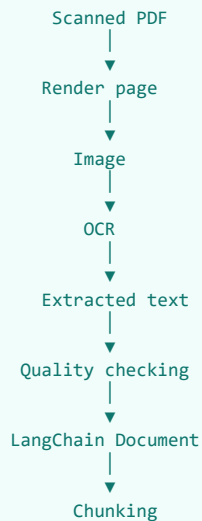
OCR Responsibility:
Image → OCR → Text

RAG Responsibility:
Text → Chunking → Embedding → Indexing → Retrieval

21. Our future implementation

When we eventually implement OCR, our pipeline will be:

PLANNED PIPELINE



Notice that quality checking sits deliberately between OCR extraction and LangChain Document creation.

22. A crucial RAG principle

OCR Principle:

OCR is an extraction mechanism, not a guarantee of document understanding.

OCR converts visual information into machine-readable text, but the resulting text may contain recognition errors, lost layout information, incorrect reading order, or lost table relationships.

Therefore, OCR output should be treated as extracted data that requires validation and appropriate downstream processing.

23. Our hands-on experiment without installing OCR

We can demonstrate the fundamental OCR pipeline input without installing an OCR engine by using PyMuPDF to render the page to an image:

PYTHON

```
page = scanned_pdf[0]

pix = page.get_pixmap(dpi=150)

pix.save("../data/raw/pdf/scanned_page.png")
```

Now verify:

PYTHON

```
from pathlib import Path

image_path = Path(
    "../data/raw/pdf/scanned_page.png"
)

print(image_path.resolve())
print("Exists:", image_path.exists())
```

Output: Exists: True

24. What did we just do?

We transformed: **Scanned PDF** → **PDF page** → **Rendered image**.
This creates the exact visual payload that OCR engines operate on:

PREPARATION FLOW

25. Inspect the generated image

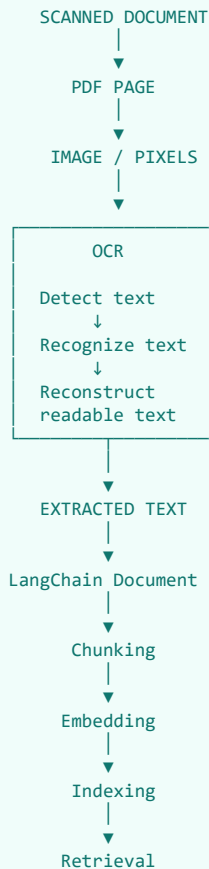
Open data/raw/pdf/scanned_page.png. You visually see text, but under the hood:

PERSPECTIVES

Human	→ sees text
PyMuPDF text extraction	→ sees no text layer
OCR (next stage)	→ sees pixels → recognizes text

26. Final mental model

COMPLETE INGESTION LIFECYCLE



Stage 1.4.2 status

STATUS ROADMAP

Stage 1.4.2 – OCR & Complex PDF Ingestion

- 1.4.2.1 Text-based PDF vs Scanned PDF
- 1.4.2.2 Detecting Scanned Pages
- 1.4.2.3 Deep PDF Page Analysis
- 1.4.2.4 OCR Fundamentals
- 1.4.2.5 OCR Implementation ← NEXT
- 1.4.2.6 OCR → LangChain Documents
- 1.4.2.7 Complex PDF / Tables / Layouts

Don't install an OCR library just yet. In **Stage 1.4.2.5**, we'll choose an OCR approach and run OCR against our `scanned_page.png`, compare the result with the original visible text, and examine limitations before building LangChain Documents.